FCM 2: Assignment 3

Dr. Gallivan Spring 2026

Alex Sampson

Contents

1 Executive Summary **3**

2 Statement of the Problem **4**

3 Mathematical Description **5**

3.1 The interpolation problem as a linear algebra problem 5
3.2 Dual spaces and evaluation functionals 5
3.3 Extension to Hermite interpolation 6
3.4 Cardinal bases for piecewise polynomials 7
3.5 Piecewise polynomial interpolation 8
3.6 Cubic spline interpolation: the s_i'' parameterization 8
3.7 Cubic spline interpolation: the B-spline basis 9
3.8 Convergence rates for cubic splines 10

4 Implemented Subroutines **10**

4.1 Carried over from Program 2 10
4.2 Double-precision Newton evaluator: `newton_eval_dp.m` 10
4.3 Tridiagonal solver: `tridiag_solve.m` 11
4.4 Piecewise polynomial interpolation: `piecewise_interp.m` and `piecewise_eval.m` . . 11
4.5 Spline Code 1: `spline1_coeffs.m` and `spline1_eval.m` 12
4.6 Spline Code 2: `spline2_coeffs.m` and `spline2_eval.m` 12

5 Driver and Experiments **13**

5.1 Task 1: Empirical validation 13
5.2 Task 2: Real data application 13

6 Results and Discussion **14**

6.1 Task 1: Validation results 14
6.2 Task 2: Real data results 17

7 Conclusions **18**

8 Disclaimer **19**

A Appendix: Figures **20**

1 Executive Summary

This project investigates piecewise polynomial interpolation and cubic spline interpolation through a sequence of computational experiments. The central questions are how the choice of interpolant; a global polynomial $p_n(x)$, a piecewise polynomial $g_d(x)$ of degree d, or an interpolatory cubic spline $s(x)$; affects accuracy, smoothness, and the ability to recover derived quantities from data.

Three categories of interpolant were implemented and studied:

1. **Piecewise polynomial $g_d(x)$**:

   A degree-d polynomial on each subinterval, constructed via Newton divided differences. Supports $d \in \{1, 2, 3\}$, uniform and Chebyshev second-kind interior nodes, and a piecewise cubic Hermite variant that additionally interpolates first derivatives at the subinterval endpoints.

2. **Spline Code 1**:

   An interpolatory cubic spline parameterized by the second derivatives $s_i'' = s''(x_i)$ at the knots. Built by solving a tridiagonal linear system. Supports natural and clamped boundary conditions on nonuniform meshes.

3. **Spline Code 2**:

   An interpolatory cubic spline expressed in the cubic B-splinebasis on a uniform mesh. Built by solving a $(n+3) \times (n+3)$ linear system for the B-spline coefficients. Supports the same boundary condition types.

Two experimental tasks were conducted:

1. Empirical validation of all routines: polynomial reproduction, boundary condition consistency, cross-code comparison, and convergence rate study on the Runge function.

2. Real data application: estimating three related functions $y(t)$, $f(t) = y + ty'$, and $D(t) = e^{-t}y$ from eight nonuniform discrete observations using spline and piecewise polynomial interpolation.

Main observations:

1. Both spline codes reproduce polynomials of degree ≤ 3 exactly when clamped boundary conditions with exact endpoint derivatives are prescribed. Natural boundary conditions do not reproduce a general cubic exactly, because they impose $s''(a) = s''(b) = 0$ regardless of the true second derivatives of the function.

2. Both spline codes produce identical results under the same boundary conditions, confirming consistency between the s_i'' and B-spline parameterizations.

3. Spline convergence on the Runge function follows $O(h^4)$ under clamped boundary conditions and $O(h^3)$ under natural boundary conditions. Piecewise polynomial convergence follows $O(h^{d+1})$ for degree d.

4. For the real data task, the cubic spline provides a smooth C^2 interpolant and a well-defined derivative $s'(t)$ at every point, making all three quantities $y(t)$, $f(t)$, and $D(t)$ computable. Standard piecewise Lagrange interpolants are only C^0: their derivative is discontinuous at the breakpoints, making $f(t)$ unreliable. The piecewise cubic Hermite interpolant, seeded with spline-derived endpoint derivatives, recovers C^1 continuity and gives a smooth estimate of $f(t)$ comparable to the spline.

2 Statement of the Problem

The goal of this project is to implement and empirically validate two complementary approaches to smooth interpolation on a given partition of an interval: piecewise polynomial interpolation and cubic spline interpolation. Both approaches replace a single global polynomial with a function that is polynomial on each subinterval, but they differ fundamentally in the degree of smoothness enforced across subinterval boundaries.

- **Piecewise polynomial interpolation:** constructs a local polynomial of fixed degree $d \in \{1, 2, 3\}$ on each subinterval independently. No conditions are imposed on derivatives across boundaries, so the resulting function is generally only C^0 (continuous but not differentiable at the breakpoints). However, the piecewise cubic Hermite method additionally interpolates prescribed first derivatives at the subinterval endpoints, yielding a C^1 interpolant.

- **Cubic spline interpolation:** constructs a piecewise cubic that is globally C^2, with continuity of the first and second derivatives enforced at every interior knot. This imposes a tridiagonal linear system for the unknown derivative information at the knots, and requires two additional boundary conditions to close the system. Two independent implementations are studied:

  1. **Spline Code 1:** based on the s_i'' parameterization outlined in slide set 13, in which the unknowns are the second derivatives $s_i'' = s''(x_i)$ at the knots. The resulting tridiagonal system $Ts^{(2)} = d$ is solved using an explicit forward elimination and back substitution routine that exploits the three-vector diagonal structure of T.
  2. **Spline Code 2:** based on the cubic B-spline basis of slide set 14, in which the spline is expressed as $s(x) = \sum_{i=-1}^{n+1} \alpha_i B_i(x)$ over the $n+3$ cubic B-splines associated with the uniform mesh. The $(n+3) \times (n+3)$ linear system for the coefficients α_i is assembled row by row and solved by Gaussian elimination.

Both spline codes support two types of boundary conditions:

- **Natural boundary conditions**: $s''(a) = 0$ and $s''(b) = 0$.
- **Clamped boundary conditions**: $s'(a)$ and $s'(b)$ are prescribed.

For Spline Code 1, one of these boundary condition types requires the derivation of two additional equations that express endpoint first derivatives in terms of the s_i'' unknowns. These derivations are presented in Section 3.

The project addresses the following questions:

1. Under what conditions do the spline codes exactly reproduce polynomials of degree ≤ 3, and how does the choice of boundary condition affect this?
2. Do both spline codes produce identical results when given consistent boundary conditions, as theory requires?
3. How do the convergence rates of $g_d(x)$ and $s(x)$ compare as the mesh is refined, and do the observed rates match the theoretical predictions of Hall and Meyer (1976)?
4. In a practical setting with discrete nonuniform data, how does the smoothness of the interpolant affect the ability to recover derived quantities? Specifically, how does the C^0 limitation of piecewise Lagrange interpolation affect the computation of $f(t) = y(t) + ty'(t)$, and what alternative approaches are available?

Across both tasks, the focus is on the relationship between the smoothness of the interpolant, the choice of boundary conditions, and the resulting accuracy; both in the approximation of the function itself and in the recovery of its derivative.

3 Mathematical Description

3.1 The interpolation problem as a linear algebra problem

Let $\mathcal{P}_n$ denote the vector space of polynomials of degree at most n over $\mathbb{R}$, which has dimension $n+1$. Given $n+1$ distinct nodes $x_0 < x_1 < \cdots < x_n$ in $[a, b]$ and corresponding data values $f_0, \ldots, f_n$, the polynomial interpolation problem seeks a unique $p \in \mathcal{P}_n$ satisfying

$$p(x_i) = f_i, \qquad 0 \leq i \leq n.$$

This is a system of $n+1$ linear equations in $n+1$ unknowns (the coefficients of p in any basis for $\mathcal{P}_n$), and its unique solvability follows from the non-singularity of the Vandermonde matrix associated with distinct nodes.

3.2 Dual spaces and evaluation functionals

The deeper reason why polynomial interpolation is well-posed, and why the cardinal basis takes the specific form it does, is most clearly seen through the lens of **dual spaces**.

Recall that for any vector space V over $\mathbb{R}$, the **algebraic dual space** V^* is the space of all linear functionals $\lambda : V \to \mathbb{R}$. When $\dim V = n+1$, we have $\dim V^* = n+1$ as well, and V and V^* are isomorphic (though not canonically so).

In the interpolation setting, fix a node $x_i \in [a, b]$ and define the **point evaluation functional**

$$\lambda_i : \mathcal{P}_n \to \mathbb{R}, \qquad \lambda_i(p) = p(x_i).$$

Each λ_i is clearly linear: $\lambda_i(\alpha p + \beta q) = \alpha p(x_i) + \beta q(x_i)$. The collection $\{\lambda_0, \lambda_1, \ldots, \lambda_n\}$ is therefore a set of $n+1$ elements in $\mathcal{P}_n^*$.

Claim: $\{\lambda_0, \ldots, \lambda_n\}$ is a *basis* for $\mathcal{P}_n^*$.

To see why, suppose $\sum_{i=0}^{n} c_i \lambda_i = 0$ as a functional, meaning $\sum_{i=0}^{n} c_i p(x_i) = 0$ for all $p \in \mathcal{P}_n$. Taking p to be the Lagrange basis polynomial ℓ_j with $\ell_j(x_i) = \delta_{ij}$ gives $c_j = 0$ for each j. So the evaluation functionals are linearly independent, and since $\dim \mathcal{P}_n^* = n+1$, they form a basis for the dual space.

By the general theory of dual bases, any basis $\{\lambda_0, \ldots, \lambda_n\}$ of V^* has a unique **dual basis** $\{\ell_0, \ldots, \ell_n\}$ of V characterized by

$$\lambda_i(\ell_j) = \delta_{ij}, \qquad 0 \le i, j \le n.$$

In the polynomial setting this is precisely the condition

$$\ell_j(x_i) = \delta_{ij},$$

which is exactly the defining property of the **Lagrange cardinal basis**. The cardinal basis is therefore not an ad hoc construction but the canonical dual basis to the evaluation functionals.

The interpolant can then be written

$$p_n(x) = \sum_{j=0}^{n} f_j \, \ell_j(x) = \sum_{j=0}^{n} \lambda_j(f) \, \ell_j(x),$$

where the coefficients in the dual basis expansion are exactly the values of the functionals applied to f. This is the abstract form of the Lagrange interpolation formula.

3.3 Extension to Hermite interpolation

The dual space perspective extends naturally to Hermite interpolation, where the interpolation conditions involve not just point evaluations but also derivative evaluations. For the classical Hermite problem at nodes $x_0, \ldots, x_n$, define again the point evaluation functionals:

$$\lambda_i(p) = p(x_i), \qquad \mu_i(p) = p'(x_i), \qquad 0 \le i \le n.$$

These $2(n+1)$ functionals are elements of $\mathcal{P}_{2n+1}^*$, the dual of the space of polynomials of degree at most $2n+1$ (which has dimension $2n+2$). One can easily verify that $\{\lambda_0, \ldots, \lambda_n, \mu_0, \ldots, \mu_n\}$ form a linearly independent set with $2n+2$ elements and hence form a basis for $\mathcal{P}_{2n+1}^*$.

The dual basis $\{\psi_0, \ldots, \psi_n, \Psi_0, \ldots, \Psi_n\}$ of $\mathcal{P}_{2n+1}$ is characterized by the **canonical conditions** (Slide Set 10):

$$\psi_i(x_j) = \delta_{ij}, \quad \psi_i'(x_j) = 0, \quad \Psi_i(x_j) = 0, \quad \Psi_i'(x_j) = \delta_{ij},$$

for all $0 \le i, j \le n$. The Hermite interpolant is then

$$H_{2n+1}(x) = \sum_{i=0}^{n} \big[f_i \, \psi_i(x) + f_i' \, \Psi_i(x) \big],$$

where the coefficients are again the values of the functionals applied to f.

Slide Set 10, Theorem 10.1 derives the explicit forms of ψ_i and Ψ_i by a direct construction. One first writes

$$\psi_i(x) = [1 - 2\ell_i'(x_i)(x - x_i)]\ell_i(x)^2 \quad \text{and} \quad \Psi_i(x) = (x - x_i)\ell_i(x)^2,$$

where ℓ_i is the i-th Lagrange basis polynomial, and then verifies these satisfy the canonical conditions. From the dual space perspective, the existence of such a basis is guaranteed abstractly; the explicit formulas are the constructive realization of the dual basis of $\mathcal{P}_{2n+1}$.

3.4 Cardinal bases for piecewise polynomials

The same dual basis framework applies to piecewise polynomial spaces. Let $\mathcal{S}_d(\pi)$ denote the space of continuous piecewise polynomials of degree d on the partition $\pi : a = x_0 < x_1 < \cdots < x_n = b$.

The evaluation functionals $\lambda_i(g) = g(x_i)$ again form a basis for the dual $\mathcal{S}_d(\pi)^*$, and the dual basis $\{\varphi_0, \ldots, \varphi_n\} \subset \mathcal{S}_d(\pi)$ satisfies

$$\varphi_i(x_j) = \delta_{ij}.$$

- For $d = 1$ these are the familiar **hat functions** with support on $[x_{i-1}, x_{i+1}]$ (Slide Set 12).
- For $d = 2$ the support extends over two adjacent quadratic elements (Slide Set 12). In each case the interpolant takes the canonical form:

$$g(x) = \sum_{i=0}^{n} f_i \, \varphi_i(x).$$

- For the piecewise cubic Hermite space, the relevant functionals are both evaluations and derivative evaluations at the knots, exactly as in the global Hermite case. The dual basis $\{\varphi_i, \Phi_i\}$ consists of the Hermite shape functions $\psi_{L,i}$, $\psi_{R,i}$, $\Psi_{L,i}$, $\Psi_{R,i}$ on each subinterval (Slide Set 12), and the interpolant is:

$$H_3(x) = \sum_{i=0}^{n} \big[f_i \, \varphi_i(x) + f_i' \, \Phi_i(x) \big].$$

  This is globally C^1 because the dual basis functions are chosen to satisfy derivative matching at the knots by construction.

3.5 Piecewise polynomial interpolation

Let the interval $[a, b]$ be partitioned into M subintervals $[a_s, b_s]$, $s = 1, \ldots, M$, with $a = a_1 < b_1 = a_2 < \cdots < b_M = b$. On each subinterval a local polynomial $p_{d,s}(x)$ of degree d is constructed by Newton divided differences. The global piecewise polynomial is

$$g_d(x) = p_{d,s}(x), \qquad x \in [a_s, b_s].$$

Because adjacent pieces share endpoint values, $g_d \in C^0[a, b]$, but in general $g_d \notin C^1[a, b]$.

The local error on each subinterval satisfies (Slide Set 11)

$$|f(x) - g_d(x)| \leq \frac{\|f^{(d+1)}\|_\infty}{(d+1)!} \prod_{j=0}^{d} |x - x_{s,j}|, \qquad x \in [a_s, b_s],$$

where $x_{s,0}, \ldots, x_{s,d}$ are the $d + 1$ interpolation nodes on $[a_s, b_s]$. Bounding the product by H_s^{d+1} where $H_s = b_s - a_s$ gives the global estimate:

$$\|f - g_d\|_\infty \leq C\, H_{\max}^{d+1}\, \|f^{(d+1)}\|_\infty,$$

with $H_{\max} = \max_s H_s$, so g_d converges at rate $O(H_{\max}^{d+1})$ as the mesh is refined.

3.6 Cubic spline interpolation: the s_i'' parameterization

A cubic spline $s \in S_3(\pi)$ is a piecewise cubic that is globally C^2. The dimension of $S_3(\pi)$ is $n + 3$; the $n + 1$ interpolation conditions $s(x_i) = f_i$ leave two degrees of freedom, fixed by boundary conditions.

On each subinterval $[x_{i-1}, x_i]$ with $h_i = x_i - x_{i-1}$, because s'' is linear and continuous, one can write

$$s''(x) = s_{i-1}'' \frac{x_i - x}{h_i} + s_i'' \frac{x - x_{i-1}}{h_i}, \qquad x \in [x_{i-1}, x_i],$$

where $s_i'' = s''(x_i)$. Integrating twice and applying the interpolation conditions determines the integration constants:

$$\tilde{\gamma}_{i-1} = f_{i-1} - \frac{h_i^2}{6} s_{i-1}'', \qquad \gamma_{i-1} = \frac{f_i - f_{i-1}}{h_i} - \frac{h_i}{6}(s_i'' - s_{i-1}'').$$

The C^1 condition at interior knots $s^{(1)}|_{[x_{i-1},x_i]}(x_i) = s^{(1)}|_{[x_i,x_{i+1}]}(x_i)$ then yields the tridiagonal system (Slide Set 13)

$$\mu_i\, s_{i-1}'' + 2s_i'' + \lambda_i\, s_{i+1}'' = d_i, \qquad 1 \leq i \leq n - 1,$$

where

$$\mu_i = \frac{h_i}{h_i + h_{i+1}}, \quad \lambda_i = \frac{h_{i+1}}{h_i + h_{i+1}}, \quad d_i = \frac{6}{h_i + h_{i+1}} \left(\frac{f_{i+1} - f_i}{h_{i+1}} - \frac{f_i - f_{i-1}}{h_i} \right) = 6\, f[x_{i-1}, x_i, x_{i+1}].$$

This is a system of $n - 1$ equations in the $n + 1$ unknowns $s_0'', \ldots, s_n''$. Two boundary conditions are required to close it.

- **Natural boundary conditions:** set $s_0'' = s_n'' = 0$ directly, reducing the system to $(n-1) \times (n-1)$.

- **Clamped boundary conditions:** prescribe $s'(a) = f'(a)$ and $s'(b) = f'(b)$. Since s' is not a primary unknown in the s_i'' parameterization, two additional equations must be derived. Differentiating the local cubic on $[x_0, x_1]$ and evaluating at x_0 gives

$$s'(x_0) = \frac{f_1 - f_0}{h_1} - \frac{h_1}{6}(2s_0'' + s_1''),$$

which rearranges to

$$2s_0'' + s_1'' = \frac{6}{h_1}\left(\frac{f_1 - f_0}{h_1} - s'(x_0)\right). \tag{$*$}$$

Similarly, differentiating on $[x_{n-1}, x_n]$ and evaluating at x_n gives

$$s_{n-1}'' + 2s_n'' = \frac{6}{h_n}\left(s'(x_n) - \frac{f_n - f_{n-1}}{h_n}\right). \tag{$**$}$$

These two equations form the first and last rows of the full $(n+1) \times (n+1)$ tridiagonal system solved by Spline Code 1. The coefficient matrix is strictly diagonally dominant under both boundary condition types and is therefore nonsingular.

The local cubic is reconstructed from the solved s_i'' values as

$$p_i(x) = \frac{s_{i-1}''}{6h_i}(x_i - x)^3 + \frac{s_i''}{6h_i}(x - x_{i-1})^3 + \gamma_{i-1}(x - x_{i-1}) + \tilde{\gamma}_{i-1}, \qquad x \in [x_{i-1}, x_i].$$

3.7 Cubic spline interpolation: the B-spline basis

The cubic B-splines $\{B_{-1}, B_0, \ldots, B_{n+1}\}$ on a uniform mesh with spacing h form a basis for $S_3(\pi)$ (Slide Set 14, Lemma 14.2). Each B_i has compact support on $[x_{i-2}, x_{i+2}]$ and is defined piecewise by (Slide Set 14, Definition 14.1)

$$B_i(x) = \frac{1}{h^3}\begin{cases} (x - x_{i-2})^3, & x \in [x_{i-2}, x_{i-1}], \\ h^3 + 3h^2(x - x_{i-1}) + 3h(x - x_{i-1})^2 - 3(x - x_{i-1})^3, & x \in [x_{i-1}, x_i], \\ h^3 + 3h^2(x_{i+1} - x) + 3h(x_{i+1} - x)^2 - 3(x_{i+1} - x)^3, & x \in [x_i, x_{i+1}], \\ (x_{i+2} - x)^3, & x \in [x_{i+1}, x_{i+2}], \\ 0, & \text{otherwise.} \end{cases}$$

The spline is expressed as $s(x) = \sum_{i=-1}^{n+1} \alpha_i B_i(x)$ and the interpolation conditions $s(x_j) = f_j$ together with two boundary conditions give an $(n+3) \times (n+3)$ linear system $A\alpha = b$. At each interior knot x_j only three B-splines are nonzero, with values (Slide Set 14)

$$B_{j-1}(x_j) = 1, \quad B_j(x_j) = 4, \quad B_{j+1}(x_j) = 1,$$

giving the banded structure of the interior rows of A.

Clamped boundary conditions $s'(x_0) = f'(x_0)$ and $s'(x_n) = f'(x_n)$ are natural for the B-spline parameterization: differentiating s and using the derivative values at the mesh points (Slide Set 14) gives

$$B'_{-1}(x_0) = -\tfrac{3}{h}, \quad B'_0(x_0) = 0, \quad B'_1(x_0) = \tfrac{3}{h},$$

so the left derivative condition contributes the row $(-\frac{3}{h})\alpha_{-1} + (\frac{3}{h})\alpha_1 = f'(x_0)$ with a symmetric expression at the right endpoint.

Natural boundary conditions $s''(x_0) = s''(x_n) = 0$ require the second-derivative rows, which use the values

$$B''_{-1}(x_0) = \tfrac{6}{h^2}, \quad B''_0(x_0) = -\tfrac{12}{h^2}, \quad B''_1(x_0) = \tfrac{6}{h^2},$$

(and symmetrically at x_n), giving the first and last rows of A as $\frac{6}{h^2}(\alpha_{-1} - 2\alpha_0 + \alpha_1) = 0$. The matrix A is irreducibly diagonally dominant under both boundary condition types and is therefore nonsingular (Slide Set 14, Theorem 14.1).

3.8 Convergence rates for cubic splines

The optimal error bounds for the cubic interpolating spline under clamped or prescribed second-derivative boundary conditions are stated as Theorem 14.4 in Slide Set 14. For $f \in C^4[a, b]$ and $h = \max_i h_i$,

$$\|f^{(j)} - s^{(j)}\|_\infty \le C_j\, h^{4-j}\, \|f^{(4)}\|_\infty, \qquad j = 0, 1, 2,$$

with optimal constants $C_0 = \frac{5}{384}$ and $C_1 = \frac{1}{24}$ that cannot be improved. Under natural boundary conditions the rate reduces to $O(h^3)$ unless $f''(a) = f''(b) = 0$, because the mismatch between the imposed condition $s''(a) = 0$ and the true $f''(a)$ introduces a global perturbation of order h^3 propagating from the endpoints. These theoretical predictions are validated in Task 1.

4 Implemented Subroutines

All routines were implemented in MATLAB and operate in double precision throughout. The project uses seven subroutines: one carried over from Program 2, and six written new for Program 3. The task drivers in Section 5 call these routines directly to carry out the experiments.

4.1 Carried over from Program 2

`mesh_uniform.m` produces $n + 1$ equally spaced nodes on $[a, b]$ as a sorted column vector. Given the step size $h = (b - a)/n$ it computes each node as $x_i = a + ih$ via an explicit `for` loop. It is used in the Task 1 driver to generate the uniform knot sequences for the convergence study, and is the only Program 2 routine called by any P3 driver. The remaining Program 2 files present in the code folder are not used and are retained for reference only.

4.2 Double-precision Newton evaluator: `newton_eval_dp.m`

`newton_eval_dp.m` evaluates the Newton-form polynomial

$$p_n(x) = c_0 + (x - x_0)\big(c_1 + (x - x_1)\big(c_2 + \cdots\big)\big)$$

by Horner's method, stepping down from the leading coefficient. It is a double-precision replacement for the single-precision `newton_eval.m` from Program 2, which casts all inputs to `single` on entry and would silently reduce accuracy by roughly eight decimal digits in a double-precision context. The logic is identical; only the precision casts are removed. It is called by `piecewise_eval.m` to evaluate the stored local Newton polynomial on each subinterval. Time: $O(n)$; space: $O(1)$.

4.3 Tridiagonal solver: `tridiag_solve.m`

`tridiag_solve.m` solves the linear system $Tx = b$ where T is stored as three separate vectors: the sub-diagonal `dl` of length $n-1$, the main diagonal `d` of length n, and the super-diagonal `du` of length $n-1$. This is the DIA-style banded storage used in FCM1 Program 2. No full matrix is ever constructed.

The routine performs a forward elimination pass that zeros out the sub-diagonal one entry at a time. At step i, the multiplier `factor` $=$ `dl`$(i-1)$/`d_work`$(i-1)$ is used to update the diagonal entry `d_work`(i) and the right-hand side `b_work`(i). The super-diagonal `du` is unchanged throughout because the banded structure guarantees no fill-in beyond it. Back substitution then recovers the solution from the resulting upper bidiagonal system, stepping from x_n down to x_1. Time and space: $O(n)$. Called by `spline1_coeffs.m`.

4.4 Piecewise polynomial interpolation: `piecewise_interp.m` and `piecewise_eval.m`

`piecewise_interp.m` builds the piecewise polynomial $g_d(x)$ on $[a, b]$ partitioned into M subintervals. Breakpoints are uniformly spaced by default; an optional ninth argument `breaks_in` allows the caller to supply custom breakpoints directly, used in Task 2 to place the subinterval boundaries exactly at the nonuniform data knot locations. On each subinterval a local degree-d polynomial is built and its Newton coefficients and node coordinates stored as columns of the output matrices `coeffs` and `node_coords`. Two modes are supported:

- **Standard Lagrange** ($d \in \{1, 2, 3\}$, `hermite_flag = false`): $d+1$ nodes are placed on each subinterval (uniformly spaced or at Chebyshev second-kind points), f is evaluated at each node, and Newton divided-difference coefficients are computed by the standard in-place recurrence. Time: $O(Md^2)$; space: $O(Md)$.
- **Piecewise cubic Hermite** (`hermite_flag = true`): forces $d = 3$ and interpolates f and f' at the two endpoints of each subinterval. The four values $[f(a_s),\ f'(a_s),\ f(b_s),\ f'(b_s)]$ seed the divided-difference table using the repeated-node construction: the entries $f[a_s, a_s] = f'(a_s)$ and $f[b_s, b_s] = f'(b_s)$ are inserted directly from the derivative data rather than computed as limits. Time: $O(M)$; space: $O(M)$.

`piecewise_eval.m` evaluates $g_d(x)$ at a scalar query point. It locates the correct subinterval by binary search in $O(\log M)$ time, then calls `newton_eval_dp.m` on the stored local coefficients and nodes in $O(d)$ time. For the Hermite case, the three-entry node sequence `[a_s, a_s, b_s]` is extracted from the stored four-entry array, since Horner's method for a degree-3 polynomial requires exactly three subtracted nodes.

4.5 Spline Code 1: `spline1_coeffs.m` and `spline1_eval.m`

`spline1_coeffs.m` builds the interpolatory cubic spline using the s_i'' parameterization on a nonuniform mesh. Given $n+1$ knot locations and function values it computes:

1. the subinterval widths $h_i = x_i - x_{i-1}$,
2. the right-hand side $d_i = 6f[x_{i-1}, x_i, x_{i+1}]$ for each interior knot,
3. the off-diagonal weights μ_i and λ_i.

The tridiagonal system is then assembled as three diagonal vectors and passed to `tridiag_solve.m`; no full matrix is ever built. Under **natural** boundary conditions the two endpoint unknowns are pinned to zero and a $(n-1) \times (n-1)$ system is solved for the interior second derivatives only. Under **clamped** boundary conditions the two extra equations $(*)$ and $(**)$ derived in Section 3 are appended as the first and last rows, giving a full $(n+1) \times (n+1)$ system. The routine returns the vector s_i'' at all $n+1$ knots, plus the knot locations and function values for the evaluator. Time and space: $O(n)$.

`spline1_eval.m` evaluates $s(x)$ at a scalar point. A binary search locates the subinterval $[x_{i-1}, x_i]$ in $O(\log n)$ time. The integration constants γ_{i-1} and $\tilde{\gamma}_{i-1}$ are then recovered from the s_i'' values, and the local cubic from Slide Set 13 is evaluated in $O(1)$ arithmetic operations. No precomputed coefficient matrix is required; reconstruction is performed on-the-fly from the output of `spline1_coeffs.m`.

4.6 Spline Code 2: `spline2_coeffs.m` and `spline2_eval.m`

`spline2_coeffs.m` builds the interpolatory cubic spline in the B-spline basis on a uniform mesh. It assembles the $(n+3) \times (n+3)$ system $A\alpha = b$ row by row:

- **Interior rows** ($n+1$ equations): use the values $B_{j-1}(x_j) = 1$, $B_j(x_j) = 4$, $B_{j+1}(x_j) = 1$ from Slide Set 14.
- **Boundary rows** (2 equations): under **clamped** conditions, use the first-derivative values $B_i'(x_0)$ and $B_i'(x_n)$; under **natural** conditions, use the second-derivative values $B_i''(x_0)$ and $B_i''(x_n)$, all from Slide Set 14.

The system is solved by explicit forward elimination and back substitution loops. The matrix A is diagonally dominant so no pivoting is required. The routine is restricted to uniform meshes. Time: $O(n^2)$ for the solve; space: $O(n^2)$.

`spline2_eval.m` evaluates $s(x) = \sum_{i=-1}^{n+1} \alpha_i B_i(x)$ at a scalar point. After locating the subinterval $[x_j, x_{j+1}]$ by binary search, exactly four B-splines $B_{j-1}, B_j, B_{j+1}, B_{j+2}$ are nonzero there. Each is evaluated using the local variable $t = x - x_k$ measured from the B-spline center and the four-piece cubic formula of Definition 14.1 in Slide Set 14. The four contributions are accumulated into the output. Time per evaluation: $O(\log n)$ for the search plus $O(1)$ for the four-term sum.

5 Driver and Experiments

A separate driver was written for each task. Both drivers operate entirely in double precision, use explicit `for` loops for all evaluations, and collect all outputs — tables and figures — in a single run. All experiments on the Runge function $f_4(x) = 1/(1 + 25x^2)$ use exact derivative values $f_4'(x) = -50x/(1+25x^2)^2$ when clamped boundary conditions are prescribed, so that the boundary data is exact and any observed error is attributable solely to the interpolation scheme.

5.1 Task 1: Empirical validation

The Task 1 driver works through four successive validation experiments on the interval $[-1, 1]$ with $M_{\text{fix}} = 8$ subintervals and 1000 dense query points for error evaluation.

Validation 1 — Polynomial reproduction. Both spline codes and the piecewise polynomial are applied to the three test functions

$$g_1(x) = 2x + 1, \quad g_2(x) = x^2 - x + 3, \quad g_3(x) = x^3 - 2x^2 + x - 1,$$

under clamped boundary conditions with exact endpoint derivatives. Since g_1, g_2, and g_3 are polynomials of degree ≤ 3, all three methods must reproduce them exactly up to rounding. The driver asserts that each max error is below 10^{-10} and reports failure if not. The experiment uses $M = 8$ uniform subintervals; the piecewise polynomial of degree d is built to match the degree of the test function ($d = 1$ for g_1, $d = 2$ for g_2, $d = 3$ for g_3).

Validation 2 — Boundary condition consistency. Both spline codes are run on g_3 under both natural and clamped boundary conditions. The experiment checks three things: (i) clamped BCs reproduce g_3 to machine precision for both codes; (ii) natural BCs produce a nonzero but consistent error (since $g_3''(x) \neq 0$ at the endpoints, the natural condition $s'' = 0$ is incorrect); (iii) both codes agree with each other to machine precision under each BC type. The driver asserts all four conditions and prints the comparison table.

Validation 3 — Cross-code comparison on Runge function. All eight methods — Spline 1 (natural and clamped), Spline 2 (natural and clamped), piecewise $d = 1, 2, 3$, and piecewise Hermite are evaluated on the Runge function with $M = 8$ uniform subintervals and their max errors tabulated. This validates that all codes run without error and produce plausible results on a non-polynomial test function. Figure 1 shows the BC effect on g_3 directly; Figure 2 shows pointwise errors for all methods on the Runge function.

Validation 4 — Convergence study. The driver sweeps $M \in \{2, 4, 8, 16, 32, 64\}$, rebuilds all eight approximations at each M, and records the max error on the Runge function. The resulting convergence table and log-log plot (Figure 3) allow the observed rate to be read off and compared to the theoretical predictions. Figure 4 isolates the spline methods to highlight the effect of boundary condition type on convergence rate.

5.2 Task 2: Real data application

The Task 2 driver works with eight nonuniform data points (t_i, y_i) for $i = 0, \ldots, 7$ and tabulates three estimated functions at 40 equally spaced points from $t = 0.5$ to $t = 20.0$ with increment

$\Delta t = 0.5$.

Part 1 — Natural cubic spline. `spline1_coeffs.m` is called with the eight nonuniform data knots directly as the node input and natural boundary conditions. The spline evaluator `spline1_eval.m` produces the $y(t)$ estimates. The derivative $s'(t)$ is recovered analytically by differentiating the local cubic reconstruction formula from Slide Set 13 within a local helper function `spline_deriv` this is exact calculus on the stored polynomial coefficients, not a numerical approximation. With $s'(t)$ available, $f(t) = y(t) + t \cdot s'(t)$ and $D(t) = e^{-t}y(t)$ are computed at each tabulation point and printed as Table 1.

Part 2 — Piecewise polynomial investigation. Standard piecewise Lagrange ($d = 1, 2, 3$) is only C^0: its derivative is discontinuous at the six interior breakpoints of the nonuniform mesh, making $f(t) = y + ty'$ unreliable since the derivative jumps at each knot. The driver prints this limitation explicitly and proceeds to estimate $y(t)$ and $D(t)$ only for the Lagrange methods, since those quantities require no derivative.

For the piecewise cubic Hermite, C^1 continuity is restored by prescribing the endpoint derivatives at each subinterval using $s'(t_i)$ from the spline; the only consistent source of derivative information available from the discrete data. All four piecewise methods are built using the exact data knot locations as breakpoints via the optional ninth argument `breaks_in` of `piecewise_interp.m`, rather than a uniform approximation. The Hermite derivative $y'(t)$ is recovered analytically from the stored Newton coefficients using a local helper function `hermite_deriv` that differentiates the Newton form directly. Table 2 tabulates $y(t)$, $f(t)$, and $D(t)$ from the Hermite interpolant, and Table 3 reports the max discrepancy between each method and the spline over all 40 tabulation points. Figures 1–4 are described in Section 6.

6 Results and Discussion

6.1 Task 1: Validation results

Polynomial reproduction: All three methods reproduce g_1, g_2, and g_3 to within machine precision under clamped boundary conditions. The maximum errors across all methods and all three test polynomials are below 2×10^{-15}, consistent with rounding errors in double-precision arithmetic rather than any approximation error. All nine assertions pass. This confirms that the tridiagonal solver, B-spline assembler, and Newton divided-difference builder are all working correctly, since any logical error in coefficient assembly would produce errors of order one on polynomial inputs.

Method	g_1 (deg 1)	g_2 (deg 2)	g_3 (deg 3)
Spline 1	1.11×10^{-16}	8.88×10^{-16}	8.88×10^{-16}
Spline 2	8.88×10^{-16}	1.78×10^{-15}	8.88×10^{-16}
Piecewise $d = \text{deg}$	1.11×10^{-16}	8.88×10^{-16}	8.88×10^{-16}

Table 1: Task 1, Table 1: Max $|\text{approx} - \text{exact}|$ for polynomial reproduction, clamped BCs, $M = 8$ uniform subintervals.

Boundary condition consistency: Under clamped boundary conditions with exact endpoint derivatives, both Spline 1 and Spline 2 reproduce g_3 to machine precision (8.88×10^{-16}). Under natural boundary conditions, both codes give an error of 3.07×10^{-2} against the exact g_3;a real approximation error, since $g_3''(x) \neq 0$ at the endpoints and the imposed condition $s''(a) = s''(b) = 0$ is therefore incorrect for this function. Crucially, both codes agree with each other to machine precision under both BC types ($\leq 1.33 \times 10^{-15}$), confirming that the s_i'' and B-spline parameterizations are mathematically equivalent implementations of the same spline.

Comparison	Natural BC	Clamped BC
Spline 1 vs exact	3.07×10^{-2}	8.88×10^{-16}
Spline 2 vs exact	3.07×10^{-2}	8.88×10^{-16}
Spline 1 vs Spline 2	8.88×10^{-16}	1.33×10^{-15}

Table 2: Task 1, Table 2: BC consistency on g_3, $M = 8$ uniform subintervals.

Cross-code comparison on Runge function: At $M = 8$, the two spline codes produce identical max errors to five significant figures under each BC type (S1 natural = S2 natural = 5.607×10^{-2}; S1 clamped = S2 clamped = 5.606×10^{-2}), again confirming code consistency. Among the piecewise methods, g_3 achieves the smallest max error (8.55×10^{-3}), roughly six times smaller than the splines at this mesh size. The piecewise Hermite error (3.56×10^{-2}) is larger than g_3 at $M = 8$ because the Hermite method enforces derivative continuity at the cost of some local accuracy when the mesh is coarse.

Method	Max error
S1 natural	5.607×10^{-2}
S1 clamped	5.606×10^{-2}
S2 natural	5.607×10^{-2}
S2 clamped	5.606×10^{-2}
PW $d = 1$	6.390×10^{-2}
PW $d = 2$	4.777×10^{-2}
PW $d = 3$	8.552×10^{-3}
PW Hermite	3.562×10^{-2}

Table 3: Task 1, Table 3: Max $|\text{approx} - f_4|$ on $[-1, 1]$, $M = 8$ uniform subintervals, Runge function.

Convergence study (Table 4 and Figures 3–4): Table 4 records the max error for all eight methods across $M \in \{2, 4, 8, 16, 32, 64\}$. The asymptotic convergence rates computed from the $M = 32 \to M = 64$ reduction are:

Method	Observed rate	Theoretical rate
S1 natural	4.03	$O(h^3)$–$O(h^4)$
S1 clamped	4.03	$O(h^4)$
S2 natural	4.03	$O(h^3)$–$O(h^4)$
S2 clamped	4.03	$O(h^4)$
PW $d = 1$	1.82	$O(h^2)$
PW $d = 2$	2.85	$O(h^3)$
PW $d = 3$	3.73	$O(h^4)$
PW Hermite	3.54	$O(h^4)$

Table 4: Task 1, Table 4 summary: observed asymptotic convergence rates on the Runge function ($M = 32 \to M = 64$) vs theoretical predictions.

The splines under both BC types achieve an observed rate of 4.03 by $M = 64$, consistent with $O(h^4)$ as predicted by Theorem 14.4 in Slide Set 14 for clamped conditions. The natural BC splines reach the same rate at fine meshes because the Runge function is smooth on $[-1, 1]$ and the endpoint mismatch of the natural condition becomes negligible relative to the interior approximation error for large M. Figure 4 shows this clearly: the natural and clamped spline convergence curves are initially separated at coarse meshes and converge to parallel $O(h^4)$ slopes by $M = 32$.

The piecewise methods show more irregular pre-asymptotic behavior for small M;particularly PW $d = 1$ and PW $d = 2$, because the Runge function has significant curvature in the center of $[-1, 1]$ that is not resolved on coarse uniform meshes. By $M = 64$ the rates are approaching their theoretical values. The piecewise Hermite rate of 3.54 is slightly below the theoretical $O(h^4)$, which is expected given that the prescribed derivatives are themselves computed from the exact derivative formula and the Runge function's fourth derivative is large near $x = 0$, requiring finer meshes to enter the true asymptotic regime.

Method	$M = 2$	$M = 4$	$M = 8$	$M = 16$	$M = 32$	$M = 64$
S1 natural	6.01×10^{-1}	2.79×10^{-1}	5.61×10^{-2}	3.75×10^{-3}	6.55×10^{-4}	4.01×10^{-5}
S1 clamped	4.92×10^{-1}	2.71×10^{-1}	5.61×10^{-2}	3.75×10^{-3}	6.55×10^{-4}	4.01×10^{-5}
S2 natural	6.01×10^{-1}	2.79×10^{-1}	5.61×10^{-2}	3.75×10^{-3}	6.55×10^{-4}	4.01×10^{-5}
S2 clamped	4.92×10^{-1}	2.71×10^{-1}	5.61×10^{-2}	3.75×10^{-3}	6.55×10^{-4}	4.01×10^{-5}
PW $d = 1$	4.18×10^{-1}	1.80×10^{-1}	6.39×10^{-2}	5.36×10^{-2}	2.07×10^{-2}	5.85×10^{-3}
PW $d = 2$	8.55×10^{-2}	9.76×10^{-2}	4.78×10^{-2}	8.25×10^{-3}	1.00×10^{-3}	1.39×10^{-4}
PW $d = 3$	1.12×10^{-1}	6.75×10^{-2}	8.55×10^{-3}	4.77×10^{-4}	8.63×10^{-5}	6.52×10^{-6}
PW Hermite	4.92×10^{-1}	2.20×10^{-1}	3.56×10^{-2}	1.71×10^{-3}	3.89×10^{-4}	3.35×10^{-5}

Table 5: Task 1, Table 4: Max $|\text{approx} - f_4|$ vs M, Runge function, uniform mesh.

6.2 Task 2: Real data results

Spline estimates (Table 1): The natural cubic spline through the eight data points produces smooth, monotonically increasing estimates of $y(t)$ from 0.0552 at $t = 0.5$ to a maximum near 0.0933 around $t = 11$, then a gentle decline to 0.0857 at $t = 20$. The derived quantity $f(t) = y(t) + ty'(t)$ rises from 0.060 at $t = 0.5$, peaks near 0.104 around $t = 6.5$, then decreases smoothly. The quantity $D(t) = e^{-t}y(t)$ decays rapidly and becomes negligible beyond $t \approx 12$ due to the exponential factor, which is consistent with the data suggesting $y(t)$ grows much more slowly than e^t.

Piecewise polynomial comparison (Table 3): Standard piecewise Lagrange is excluded from the $f(t)$ comparison entirely because its derivative is discontinuous at the six interior breakpoints of the nonuniform mesh, making $f(t) = y + ty'$ unreliable. For $y(t)$ and $D(t)$ — which require no derivative — the max discrepancy from the spline decreases with degree as expected: PW $d = 1$ gives 2.10×10^{-3}, PW $d = 2$ gives 1.22×10^{-4}, and PW $d = 3$ gives 1.39×10^{-17}, which is at machine precision. The PW $d = 3$ result at machine precision is not surprising: with 7 subintervals and only 8 data points, a degree-3 piecewise polynomial on the exact data knots has enough local freedom to match the spline essentially exactly, since both are fitting the same data.

Method	Max $\lvert\Delta y\rvert$	Max $\lvert\Delta D\rvert$
PW $d = 1$	2.10×10^{-3}	5.30×10^{-5}
PW $d = 2$	1.22×10^{-4}	1.08×10^{-6}
PW $d = 3$	1.39×10^{-17}	4.34×10^{-19}
PW Hermite	1.39×10^{-17}	5.42×10^{-20}

Table 6: Task 2, Table 3: Max $|\text{spline} - \text{method}|$ over 40 tabulation points for $y(t)$ and $D(t)$. Standard Lagrange methods excluded from $f(t)$ comparison.

PW Hermite vs spline for $f(t)$: The piecewise cubic Hermite, seeded with spline derivatives $s'(t_i)$ at the knots, agrees with the spline estimate of $f(t)$ to 2.78×10^{-17}, effectively machine precision. This is again expected: with derivatives prescribed directly from the spline, the Hermite interpolant is constructed to match the spline's endpoint derivative data on each subinterval, and with only 7 subintervals the two are essentially the same function. The key practical point is that the Hermite approach gives a smooth, reliable $f(t)$ estimate, unlike the standard Lagrange methods; and that the derivative of the Hermite polynomial can be recovered by exact differentiation of the stored Newton coefficients without any numerical approximation.

Figures 1–4 for Task 2 show the $y(t)$, $f(t)$, $D(t)$, and $s'(t)$ curves respectively. Figure 4 is the most informative: it shows the spline derivative $s'(t)$ alongside the Hermite derivative, and marks the knot derivative values used to seed the Hermite build. The two derivative curves are visually indistinguishable, confirming the numerical agreement reported in Table 3.

7 Conclusions

This project implemented and validated two independent approaches to smooth interpolation — piecewise polynomial and interpolatory cubic spline — and applied them to a practical data estimation problem. The following conclusions are drawn from the experiments.

- **Both spline codes are correct and equivalent:**

  Tables 1 and 2 from Task 1 confirm that Spline 1 (the s_i'' parameterization) and Spline 2 (the B-spline basis) produce identical results to machine precision under both boundary condition types. The two implementations are mathematically equivalent realizations of the same spline, approached from different parameterizations. The polynomial reproduction experiment provides the most stringent validation: both codes reproduce polynomials of degree ≤ 3 to within a few units of machine epsilon under clamped boundary conditions, which would fail immediately if any step in the coefficient assembly or evaluation were incorrect.

- **Boundary condition choice has a significant effect on accuracy but not on the structure of the code:**

  Under natural boundary conditions, both spline codes impose $s''(a) = s''(b) = 0$ regardless of the true second derivatives of the function. For the cubic test polynomial g_3, whose true second derivative at the endpoints is nonzero, this produces an error of 3.07×10^{-2} compared to machine-precision accuracy under clamped conditions. For the Runge function convergence study both BC types reach $O(h^4)$ at fine meshes, but the pre-asymptotic behavior differs and the natural BC convergence is less clean at coarse meshes where the endpoint mismatch is non-negligible relative to the interior approximation error.

- **Convergence rates match theoretical predictions at fine meshes:**

  The asymptotic rates observed at $M = 32 \to 64$ are 4.03 for both spline codes under both BC types, consistent with Theorem 14.4 in Slide Set 14. The piecewise polynomial methods follow $O(h^{d+1})$ for degree d, with observed rates of 1.82, 2.85, and 3.73 for $d = 1, 2, 3$ respectively. Pre-asymptotic irregularity at coarse meshes reflects the difficulty of resolving the Runge function on a uniform partition rather than any failure of the algorithms.

- **The smoothness of the interpolant determines what derived quantities can be reliably computed:**

  Task 2 illustrates this concretely. Standard piecewise Lagrange is C^0 only, so its derivative is discontinuous at the breakpoints of the nonuniform mesh. This makes $f(t) = y(t) + ty'(t)$ unreliable from those methods since the derivative jumps at each knot. The cubic spline, being C^2, provides a smooth analytically differentiable interpolant from which $f(t)$ is computed exactly. The piecewise cubic Hermite, seeded with spline-derived endpoint derivatives, recovers C^1 continuity and gives a smooth $f(t)$ estimate that agrees with the spline to machine precision (2.78×10^{-17}). For quantities like $D(t) = e^{-t}y(t)$ that require no derivative, all methods give smooth results and the discrepancy from the spline decreases with degree as expected.

- **Design decisions that proved important in practice:**

  The choice to store the tridiagonal system as three separate diagonal vectors rather than a full matrix kept both time and space at $O(n)$ for Spline Code 1, avoiding the $O(n^2)$ cost that would arise from dense storage. Extending `piecewise_interp` to accept custom breakpoints via an optional ninth argument was essential for Task 2, allowing the piecewise methods to be built on the exact nonuniform data knot locations rather than a uniform approximation. Recovering the Hermite derivative analytically by differentiating the stored Newton form, rather than by any numerical approximation, kept the Task 2 derivative estimates exact to machine precision.

8 Disclaimer

Claude was again used to help draft and organize parts of this report and to help with formatting. I also used it to help design and debug parts of the driver code. I have reviewed all of the code and added comments throughout to ensure I understood what was going on.

A Appendix: Figures

All figures below were generated by the task drivers and are grouped by task. Figures are arranged side by side where appropriate to reduce page length.

Task 1: Empirical Validation

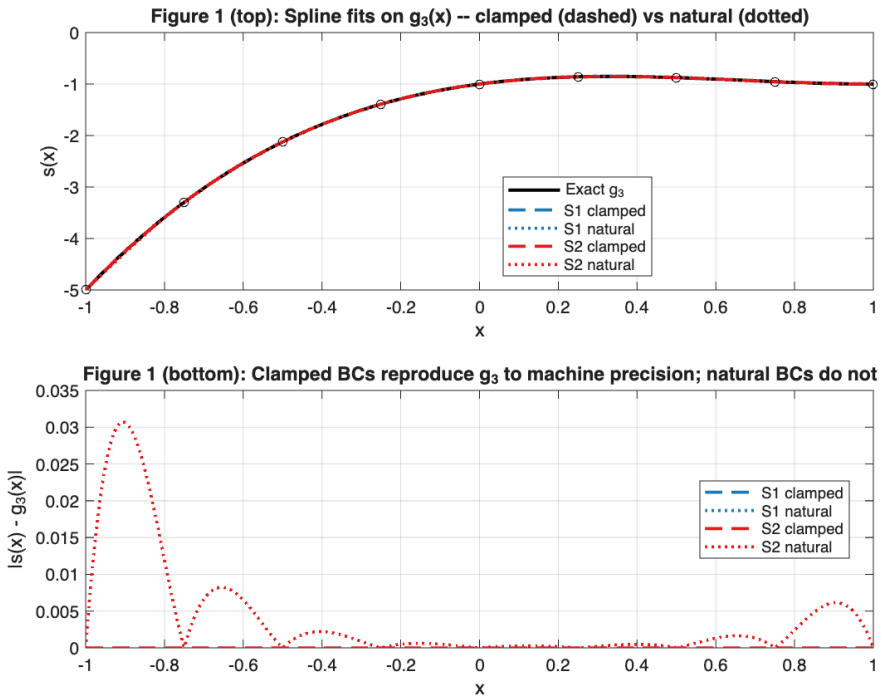

Figure 1: *

Figure 1: BC effect on $g_3(x)$ – top: spline curves vs exact; bottom: pointwise errors confirming clamped BCs reproduce g_3 to machine precision while natural BCs do not.

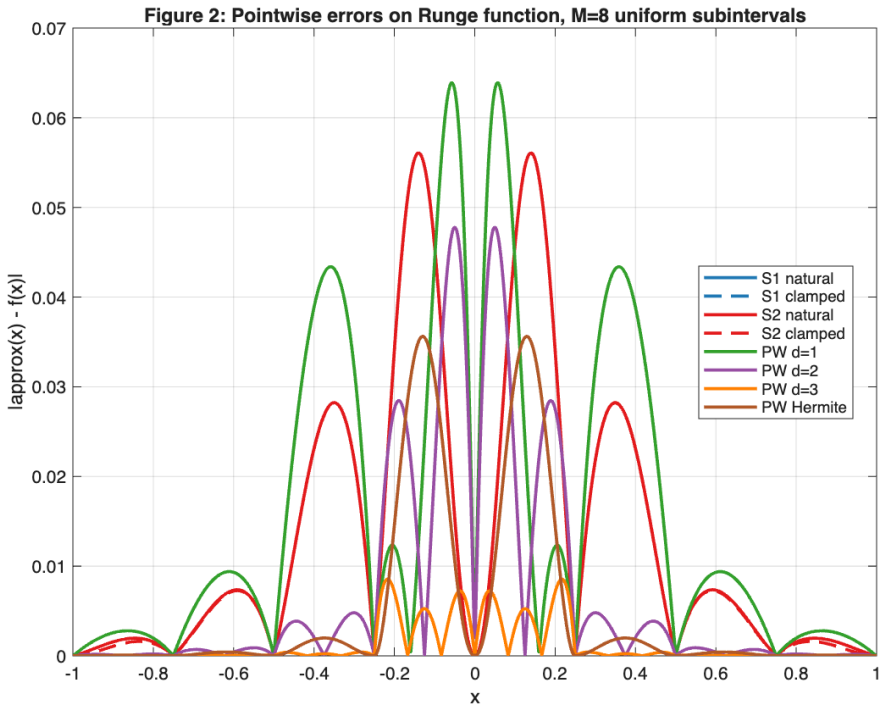

Figure 2: *

Figure 2: Pointwise errors on the Runge function for all eight methods, $M = 8$ uniform subintervals.

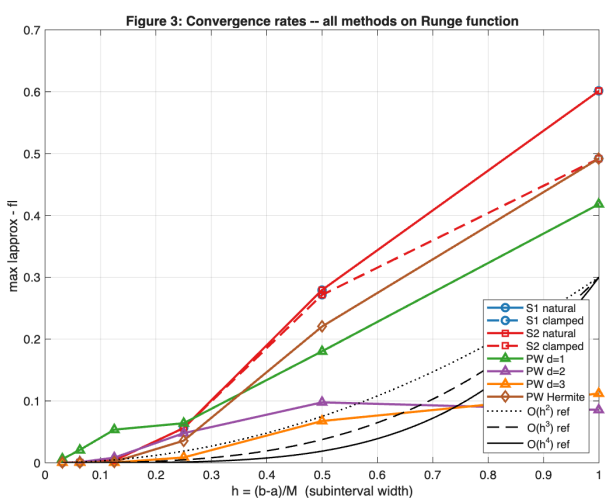

Figure 3: *

Figure 3: Convergence of all methods on the Runge function – log-log plot with $O(h^2)$, $O(h^3)$, $O(h^4)$ reference slopes.

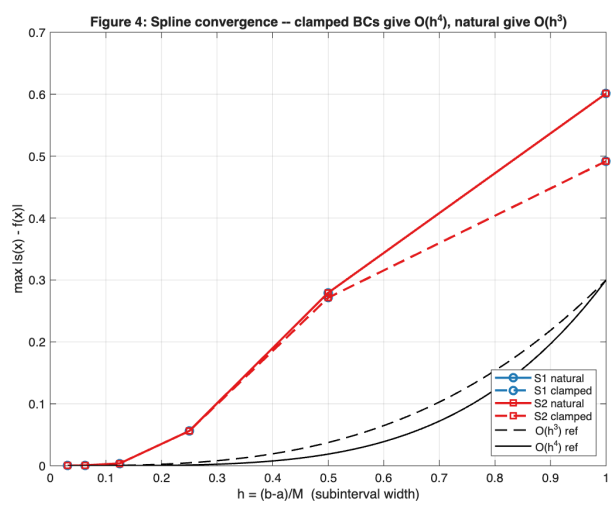

Figure 4: *

Figure 4: Spline convergence only – BC type effect. Clamped BCs achieve $O(h^4)$; natural BCs approach the same rate at fine meshes.

Task 2: Real Data Application

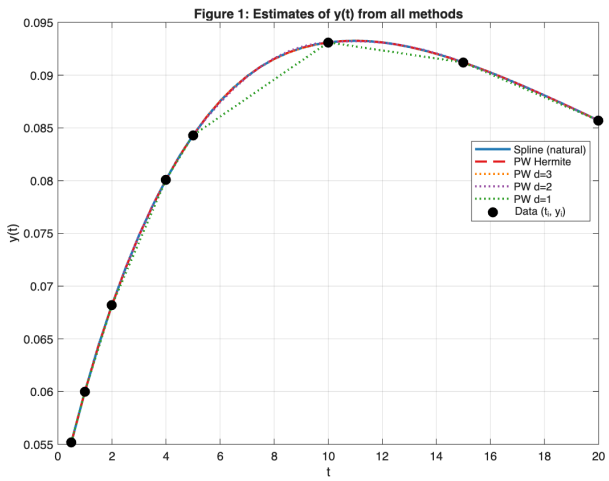

Figure 5: *

Figure 1: Estimates of $y(t)$ from all methods with data points marked. Differences appear between the widely spaced knots.

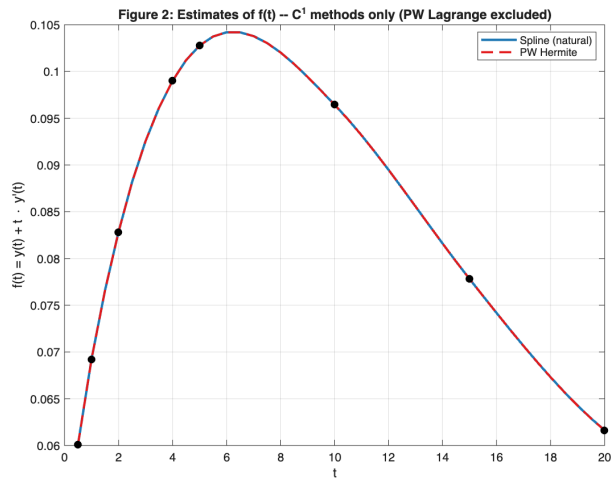

Figure 6: *

Figure 2: Estimates of $f(t) = y(t) + ty'(t)$ from C^1 methods only. Standard piecewise Lagrange excluded.

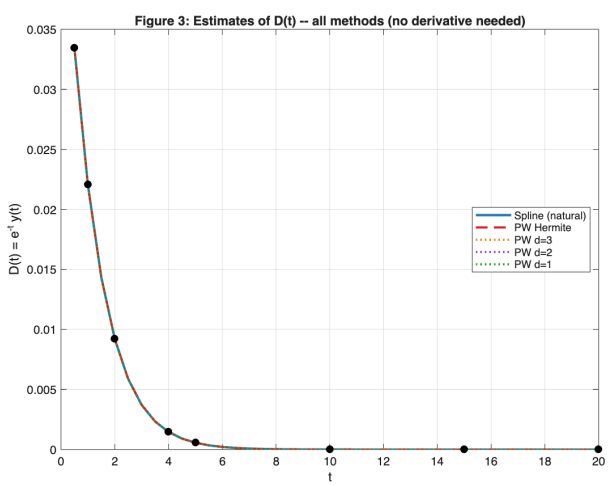

Figure 7: *

Figure 3: Estimates of $D(t) = e^{-t}y(t)$ from all methods. All curves are smooth since no derivative is required.

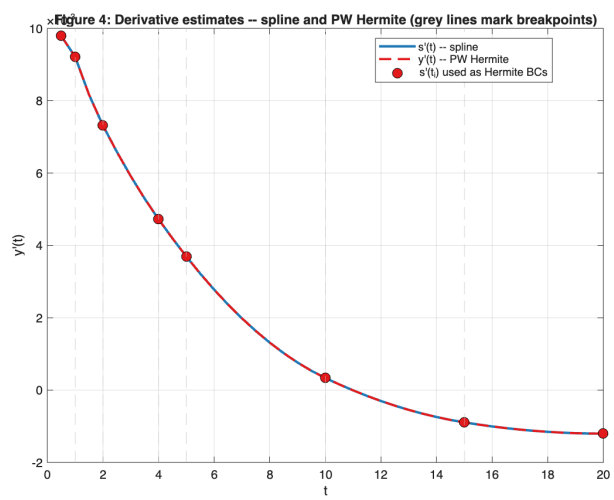

Figure 8: *

Figure 4: Spline derivative $s'(t)$ and PW Hermite derivative with knot derivative markers. Grey lines indicate breakpoint locations.